DAD-M-Methodik-Audit

Nutzen, Schwachstellen und Überarbeitungsbedarf

Datum: 14. Oktober 2026 **Gegenstand:** Bewertung des DAD-M-Frameworks und seiner praktischen Nutzung im Repository roll-drauf-vtt **Phase:** Discover-Audit; keine Änderung der Methodik oder des Produktcodes

Kurzfazit

DAD-M besitzt eine starke Kernidee, wird im Repository derzeit aber überzeugender beschrieben als operationalisiert.

Das Urteil in Kürze:

- Für risikoreiche, systemübergreifende und produktionsnahe Arbeiten ist DAD-M sehr nützlich.
- Als Standardverfahren für gewöhnliche Änderungen erzeugt es zu viel Dokumentationsaufwand.
- Die Trennung von Fakten, Entwurf, Umsetzung und Validierung ist konzeptionell klar und wertvoll.
- Die praktische Umsetzung ist bei Artefakten, Risikoübergängen, Meilenstein-Identitäten und der Codex-Steuerung inkonsistent.
- DAD-M sollte weiterentwickelt, nicht verworfen werden.

Was an DAD-M tatsächlich nützlich ist

Der größte Nutzen entsteht durch die bewusste Trennung von Faktenaufnahme, Lösungsentwurf, Umsetzung und nachgelagerter Validierung. Diese Trennung reduziert vorschnelle Implementierungen und macht produktionsnahe Arbeit leichter überprüfbar.

Der jüngste Ablauf zur Wiederherstellung ausgeschiedener Kampagnenmitglieder ist ein gutes Beispiel:

- Apply definiert Verträge, Grenzen und Abnahmekriterien.
- Deploy dokumentiert Rollback-Punkte, Tests und eine tatsächlich aufgetretene Abweichung zwischen Entwicklungs- und Produktionssystem.
- Monitor trennt das behobene Verhalten sauber von verbleibenden, unabhängigen Risiken.

Weitere starke Elemente sind:

- explizite Scope-Grenzen und Nicht-Ziele;
- Abnahmekriterien vor der Umsetzung;
- Verifikation mit konkreten Befehlen, Fehlern und offenen Lücken;
- menschliche Entscheidungspunkte für Sicherheits-, Datenschutz-, Abhängigkeits- und strategische Architekturfragen;
- vorsichtige, evidenzgerechte Formulierungen;
- die ehrliche Feststellung, dass die zentrale DAD-M-Annahme noch nicht empirisch belegt ist;
- die ausdrückliche Erkenntnis, dass Kleinstaufgaben keinen vollständigen Zyklus rechtfertigen.

Evidenzbeispiele:

- docs/DADM_APPLY_CAMPAIGN_DEPARTURE_RECOVERY_2026-10-13.md
- docs/DADM_DEPLOY_CAMPAIGN_DEPARTURE_RECOVERY_2026-10-13.md
- docs/DADM_MONITOR_CAMPAIGN_DEPARTURE_RECOVERY_2026-10-13.md
- dadm/framework/docs/methodology.md

- dadm/framework/docs/use-cases.md

Auch die Framework-Dateien selbst sind formal grundsätzlich gesund: Alle elf YAML-Dateien, alle Agent-TOML-Dateien und die Registry-JSON konnten geparst werden. Bei einer Prüfung von 59 relativen Markdown-Verweisen im Framework wurde kein defekter Link gefunden.

Wesentliche Befunde

1. Hohe Priorität: Die BIOS-Steuerung ist veraltet und der Generator arbeitet nach dem Repository-Umzug nicht mehr korrekt

Die gespeicherte Registry verweist weiterhin auf die frühere Struktur dadm-framework, .agents/skills und .codex/agents. Die aktuelle Struktur verwendet dagegen dadm/framework, dadm/skills und dadm/agents.

Eine schreibgeschützte Ausführung der Generatorlogik ermittelte dadm/ als Repository-Wurzel und fand dadurch null Policies, null Skills und null Agents. Damit ist ausgerechnet die Komponente, die Fähigkeiten und Regeln zuverlässig auffindbar machen soll, selbst nicht mehr zuverlässig.

Evidenz:

- dadm/framework/runtime/bios.registry.json
- dadm/framework/runtime/build_bios_registry.py
- AGENTS.md

2. Hohe Priorität: Die Artefaktaufbewahrung widerspricht dem tatsächlichen Repository-Verhalten

Geschlossene Phasenausgaben sind laut Methodik unveränderliche Belege. Gleichzeitig bezeichnet .gitignore Meilenstein-Ausgaben als flüchtig. Von 320 Dateien unter dadm/artifacts werden 112 ignoriert.

Das schwächt genau jene Nachvollziehbarkeit, die DAD-M als Kernnutzen verspricht. Ein Artefakt kann nicht gleichzeitig unveränderlicher Nachweis und bewusst nicht versionierter Wegwerfbestand sein.

Evidenz:

- dadm/framework/framework/core/artifact-retention.md
- .gitignore

3. Hohe Priorität: Die Übergangsregel für mittlere und hohe Risiken erzeugt einen methodischen Deadlock

Discover darf keine Lösung entwerfen. Gleichzeitig dürfen offene Risiken ab Stufe medium nicht in Apply übernommen werden, sofern sie nicht gelöst oder akzeptiert wurden.

Viele Risiken können aber erst im Lösungsentwurf behandelt werden. Die Methodik benötigt deshalb eine Unterscheidung zwischen:

- einem ungeklärten, tatsächlich blockierenden Risiko;
- einer bekannten Entwurfsbedingung mit klarer Zuständigkeit;
- einem bewusst akzeptierten Risiko;
- einem in ein anderes Meilenstein übertragenen Risiko.

Ohne diese Unterscheidung werden Risiken entweder künstlich als „gelöst“ bezeichnet oder die Methodik blockiert den Schritt, in dem die Lösung überhaupt entworfen werden dürfte.

Evidenz:

- dadm/framework/framework/core/phases.md
- dadm/framework/governance/severity-framework.md

4. Mittlere Priorität: Deklariertes und tatsächlich verwendetes Artefaktschema unterscheiden sich

Das kanonische Aufbewahrungsvokabular lautet immutable, versioned und mutable. Neuere Artefakte verwenden dagegen unter anderem review, source, rollback und deployment_rollback.

Eine heuristische Prüfung von 241 phasenbezogenen Artefaktdateien ergab:

- 235 enthalten einen Status;
- 89 enthalten eine Input-Zusammenfassung;
- 119 nennen Artefakte;
- 57 enthalten einen Abschnitt zu Risiken und Annahmen;
- 121 nennen einen nächsten Schritt;
- 29 erwähnen eine Aufbewahrungsklasse;
- nur fünf erwähnen ein Entscheidungsprotokoll.

Die Methodik verlangt dagegen für jedes Meilenstein ein fortlaufendes Entscheidungsprotokoll. Das Problem liegt daher nicht nur in einzelnen alten Dokumenten, sondern in der praktischen Umsetzbarkeit und Durchsetzung des Schemas.

5. Mittlere Priorität: Die Artefaktmenge verdeckt den aktuellen Stand

DAD-M umfasst im Repository derzeit ungefähr:

- 404 Dateien;
- 45.000 Zeilen;
- 2,6 MB Daten;
- 320 Dateien allein unter dadm/artifacts.

Zwischen docs/ und dadm/artifacts/ existieren 152 Gruppen exakt identischer Dateien mit insgesamt 348 Dateikopien. Zwei jüngere vollständige Abläufe umfassen zusammen bereits ungefähr 820 Zeilen Phasendokumentation.

Diese Menge verbessert die Nachvollziehbarkeit nicht automatisch. Ohne eindeutige Autorität und einen aktuellen Index wird Dokumentation selbst zum Suchproblem.

6. Mittlere Priorität: Meilenstein-IDs sind nicht repositoryweit eindeutig

Im Repository existieren überlappende Bereiche wie M01–M20, M17–M36, M63–M68, zusätzliche README-Meilensteine und informelle Nummern in Commit-Nachrichten. Unterschiedliche Vorhaben verwenden dieselbe Nummer für unterschiedliche Inhalte.

Die repositoryeigene Gap-Analyse kam bereits im Juli zu dem Ergebnis, dass die Frage nach dem „aktuellen Meilenstein“ keine einzige richtige Antwort besitzt.

Evidenz:

- docs/vtt_gap_analysis_2026_07_30.md
- docs/VTT_STABILIZATION_MATURITY_MATRIX_2026-07-30.md

7. Mittlere Priorität: Monitor-Abschluss und Restrisiko-Regeln widersprechen einander

Die Methodik erklärt ungelöste Risiken ab medium zu Abschlussblockern. Praktische Monitor-Dokumente schließen Meilensteine trotzdem, wenn solche Risiken außerhalb des Meilensteins liegen und als separates Folgevorhaben behandelt werden sollen.

Dieses praktische Verhalten ist sinnvoll. Das formale Risikomodell kann es aber derzeit nicht sauber ausdrücken. Es benötigt mindestens Status, Zuständigkeit, Begründung, Zielmeilenstein und gegebenenfalls eine Ablauffrist für die Akzeptanz.

8. Mittlere Priorität: Die Profile verändern hauptsächlich die geladene Dokumentationsmenge, nicht den Prozessumfang

fast, development, standard und strict verändern vor allem, welche Runtime-Karten geladen werden. Die grundsätzliche Erwartung eines vollständigen Vier-Phasen-Zyklus bleibt bestehen.

Dadurch sind die Profile als echte Betriebsmodi nur eingeschränkt nützlich. Ein Fast-Modus sollte nicht nur weniger Methodik lesen, sondern für risikoarme Arbeit auch weniger Artefakte verlangen.

9. Mittlere Priorität: Es fehlt ein Wirksamkeits-Feedback für die Methodik selbst

DAD-M dokumentiert Projektrisiken ausführlich, misst aber nicht, ob DAD-M selbst:

- Fehler früher entdeckt;
- Nacharbeit reduziert;
- Übergaben verbessert;
- Produktionsregressionen verhindert;
- oder hauptsächlich Dokumentationsaufwand erzeugt.

Die zentrale Designannahme bleibt damit trotz umfangreicher Nutzung unbewertet.

10. Niedrige Priorität: Teile des Frameworks sind eher Zielbild als gelebte Funktion

Modulschnittstellen, Modulregistries, Software-Compliance, automatische Durchsetzung und unveränderliche Speicherung sind ausführlich beschrieben. Im Framework existiert jedoch nur ein Automatisierungsskript und keine Konformitäts-Testsuite.

Diese Elemente sollten eindeutig als umgesetzt, experimentell oder zurückgestellt gekennzeichnet werden.

11. Niedrige Priorität: Der V1-Freigabedatensatz ist in sich mehrdeutig

Der Datensatz bezeichnet die Phase als abgeschlossen, nennt als nächsten Schritt aber die noch einzuholende Freigabe. Gleichzeitig enthält er status: approved und protokolliert die Version an anderer Stelle als weiterhin freigabeabhängigen Kandidaten.

Das zeigt, dass Vorschlag, Entscheidung, Phasenabschluss und Release-Status im aktuellen Artefaktmodell nicht klar genug getrennt sind.

Evidenz:

- dadm/framework/docs/decisions/M20_final_comparison_release.md

Überarbeitungspunkte für eine Apply-Phase

1. Risikogestufte Ausführung

Es sollte entschieden werden, ob risikoarme Arbeiten mehrere Phasen in einem gemeinsamen Lebenszyklus-Dokument abbilden dürfen, solange die gedanklichen Grenzen erhalten bleiben. Getrennte Discover-, Apply-, Deploy- und Monitor-Artefakte sollten vor allem für risikoreiche, umfangreiche oder sitzungsübergreifende Arbeiten verpflichtend sein.

2. Eindeutige Risiko-Disposition

Das Risikomodell sollte Zustände wie open-blocking, accepted, design-constraint, transferred, deferred-to-milestone und resolved abbilden. Ergänzend werden Verantwortliche, Begründung, Zieltermin oder Zielmeilenstein und Evidenz benötigt.

3. Eine einzige Artefaktautorität

Es muss entschieden werden, ob docs/ oder dadm/artifacts/ die verbindlichen Lebenszyklus-Datensätze enthält. Identische Kopien in mehreren Sitzungsordnern sollten vermieden werden. Ebenso muss eindeutig festgelegt werden, welche Ausgaben versioniert werden und welche bewusst temporär bleiben.

4. Ein gemeinsames Lebenszyklus-Schema

Status-, Aufbewahrungs-, Freigabe- und Entscheidungsvokabulare müssen zwischen Runtime-Karten, Templates, Policies, aktuellen Artefakten und Software-Referenz vereinheitlicht werden. Für kompakte Arbeiten sollte ein fortlaufendes Lebenszyklus-Dokument statt vier weitgehend wiederholender Dateien geprüft werden.

5. Namensräume für Meilensteine

Statt nackter Bezeichnungen wie M20 sollten Identitäten wie FRAME-M20, BOOK-M20 oder eine stabile Programm-ID mit Anzeigenummer verwendet werden. Zusätzlich wird ein maschinenlesbarer Index für den aktuellen Stand benötigt.

6. Ausführbare Konformitätsprüfung

CI oder ein lokaler Prüfbefehl sollte mindestens Folgendes validieren:

- Aktualität der BIOS-Registry;
- Artefaktschema und zulässige Aufbewahrungsklassen;
- gültige Phasenübergänge;
- doppelte autoritative Artefakte;
- veraltete Pfade und Querverweise;
- eindeutige Meilenstein-Identitäten.

Das Framework sollte die Art von Drift erkennen, die es verhindern soll.

7. Klare Behandlung von Abweichungen während Deploy

Die Methodik sollte festlegen, wann eine Implementierungskorrektur noch begrenzte Deploy-Nacharbeit ist und wann eine Entwurfsänderung vorliegt, die zwingend nach Apply zurückführt.

8. Wirksamkeitsprüfung der Methode

Sinnvolle, leichte Kennzahlen wären:

- Rücksprünge zwischen Phasen;
- vor Deploy entdeckte Fehler;
- entkommene Regressionen;
- Wartezeit auf Freigaben;

- Wiederverwendung von Artefakten;
- Aufwand für Erstellung und Pflege der Dokumentation.

Regeln ohne nachweisbaren Nutzen sollten regelmäßig vereinfacht oder entfernt werden.

9. Normativen Kern und experimentelle Erweiterungen trennen

Der kleine operative Kern sollte eindeutig autoritativ bleiben. Modulsystem, Software-Compliance, Education-Variante und BIOS-Routing sollten sichtbar als produktiv, experimentell oder zurückgestellt klassifiziert werden.

Offene Entscheidungen

Vor einer Änderung des Frameworks sind insbesondere folgende Entscheidungen erforderlich:

1. Ist DAD-M primär eine öffentliche, allgemeine Methode oder ein für dieses Repository optimiertes Betriebsmodell?
2. Dürfen risikoarme Arbeiten ein zusammengefasstes Lebenszyklus-Dokument verwenden?
3. Welche Lebenszyklus-Artefakte müssen in Git aufgenommen werden?
4. Sind Modul- und Software-Referenzverträge aktive Roadmap oder konzeptionelles Material?
5. Werden historische Artefakte migriert, als Legacy indexiert oder unverändert belassen?

Empfohlener nächster Schritt

Als nächster begrenzter Schritt sollte eine Apply-Phase „DAD-M v1.1 – Kohärenz und Skalierung“ durchgeführt werden.

Sie sollte vor jeder Framework-Änderung verbindlich festlegen:

- die normative Quelle der Wahrheit;
- das Risiko- und Übergangsmodell;
- echte Ausführungsmodi für unterschiedliche Risikostufen;
- das gemeinsame Artefaktschema;
- den Meilenstein-Namensraum;
- die Grenze einer Migration bestehender Artefakte;
- die Abnahmekriterien für eine automatische Konformitätsprüfung.

Prüfrahmen

Die zugrundeliegende Prüfung erfolgte ausschließlich lesend. Bestehende Änderungen im Worktree wurden nicht verändert. Die Bewertung stützt sich auf Framework-Dokumente, Runtime-Karten, Skills, Agent-Definitionen, Registry, Artefaktbestand sowie repräsentative aktuelle Discover-, Apply-, Deploy- und Monitor-Dokumente.